Control plane components

kube-apiserver:

- Acts as the central management entity for the Kubernetes cluster.
- Exposes the Kubernetes API, which is used by users, administrators, and other components to interact with the cluster.

etcd:

- Consistent and highly available key-value store used as the Kubernetes cluster's backing store for all cluster data.
- Stores configuration data, state information, and metadata.

kube-scheduler:

- Watches for newly created pods with no assigned node and selects a node for them to run on.
- Considers factors like resource requirements, hardware or software constraints, affinity and anti-affinity specifications, data locality, and more.

kube-controller-manager:

- Runs controller processes to regulate the state of the system.
- Controllers manage different aspects of the cluster, such as nodes, replication, endpoints, and more.

cloud-controller-manager:

- Extends the functionality of the cluster by integrating with cloud provider APIs.
- Manages external cloud services like load balancers, storage volumes, and networking.

Node components

kubelet:

- Acts as an agent running on each node in the cluster.
- Ensures that containers are running in a pod.

kube-proxy:

- Maintains network rules on nodes.
- Enables communication across pods in a cluster.

Container runtime:

- Responsible for pulling and running container images.
- Kubernetes supports multiple container runtimes, such as Docker, containerd, and others.

Add-ons

DNS server (kube-dns or CoreDNS):

- Provides DNS-based service discovery for services within the cluster.

Dashboard:

- Web-based user interface for managing and monitoring the cluster.

Ingress controller:

- Manages external access to services within a cluster, typically handling load balancing, SSL termination, and routing.

Heapster (optional, deprecated):

- Collects and interprets cluster-wide node and pod resource usage data.

Prometheus/Grafana (monitoring):

- Tools for monitoring the health and performance of the cluster.

Fluentd/ELK Stack (logging):

- Manages and collects logs from the various components in the cluster.

CNI plugins:

- Container network interface (CNI) plugins enable pod-to-pod communication and networking.

The deployment of core components

Pods: In container orchestration, a pod is the smallest deployable unit. It is the basic building block and represents a single instance of a running process in a cluster. However, unlike traditional standalone containers, pods can contain one or more containers that share the same network name space and storage, and have the capability to communicate with each other using localhost. This enables tightly coupled applications to run together in the same pod.

Nodes: Nodes are individual machines (virtual or physical) in a cluster that run your applications. Each node is responsible for running pods and providing the necessary runtime environment, including containers, necessary libraries, and other resources. They communicate with the control plane and are the workhorses of the cluster, executing the tasks assigned to them.

Clusters: A cluster is a collection of nodes that work together to run containerised applications. It includes the control plane, which manages and monitors the cluster, and the nodes, where the applications run. Clusters provide the infrastructure needed to deploy, scale, and manage containerised applications effectively. They offer high availability and reliability by distributing workloads across multiple nodes.

Deployments: Deployments are a higher-level abstraction that enables declarative updates to applications. They allow you to describe the desired state for your application, such as the number of replicas (pod instances) and the desired pod template. The deployment controller then takes care of creating, updating, and scaling the pods to match the desired state. Deployments make it easy to manage the rollout of new features, updates, or rollbacks in a controlled and automated manner.

Why use Kubernetes?

Kubernetes has become the *de facto* standard for container orchestration. It provides a robust and scalable platform for deploying, managing, and scaling containerised applications. Kubernetes brings order and efficiency to the complex task of container orchestration, enabling organisations to harness the full potential of container technology.

Challenges in traditional deployment

In traditional deployment models, managing and scaling applications can be a daunting task, including challenges like:

- ***Manual scaling:*** Scaling applications manually is time-consuming and prone to errors.